

Chapter

3

The Relational Model

Chapter Objectives

In this chapter you will learn:

- The origins of the relational model.
- The terminology of the relational model.
- How tables are used to represent data.
- The connection between mathematical relations and relations in the relational model.
- Properties of database relations.
- How to identify candidate, primary, alternate, and foreign keys.
- The meaning of entity integrity and referential integrity.
- The purpose and advantages of views in relational systems.

The Relational Database Management System (RDBMS) has become the dominant data-processing software in use today, with estimated new licence sales of between US\$6 billion and US\$10 billion per year (US\$25 billion with tools sales included). This software represents the second generation of DBMSs and is based on the relational data model proposed by E. F. Codd (1970). In the relational model, all data is logically structured within relations (tables). Each **relation** has a name and is made up of named **attributes** (columns) of data. Each **tuple** (row) contains one value per attribute. A great strength of the relational model is this simple logical structure. Yet, behind this simple structure is a sound theoretical foundation that is lacking in the first generation of DBMSs (the network and hierarchical DBMSs).

We devote a significant amount of this book to the RDBMS, in recognition of the importance of these systems. In this chapter, we discuss the terminology and basic structural concepts of the relational data model. In the next chapter, we examine the relational languages that can be used for update and data retrieval.

Structure of this Chapter

To put our treatment of the RDBMS into perspective, in Section 3.1 we provide a brief history of the relational model. In Section 3.2 we discuss the underlying concepts and terminology of the relational model. In Section 3.3 we discuss the relational integrity rules, including entity integrity and referential integrity. In Section 3.4 we introduce the concept of views, which are important features of relational DBMSs although, strictly speaking, not a concept of the relational model *per se*.

Looking ahead, in Chapters 5 and 6 we examine SQL (Structured Query Language), the formal and *de facto* standard language for RDBMSs, and in Chapter 7 we examine QBE (Query-By-Example), another highly popular visual query language for RDBMSs. In Chapters 15–18 we present a complete methodology for relational database design. In Appendix D, we examine Codd’s twelve rules, which form a yardstick against which RDBMS products can be identified. The examples in this chapter are drawn from the *DreamHome* case study, which is described in detail in Section 10.4 and Appendix A.

3.1 Brief History of the Relational Model

The relational model was first proposed by E. F. Codd in his seminal paper ‘A relational model of data for large shared data banks’ (Codd, 1970). This paper is now generally accepted as a landmark in database systems, although a set-oriented model had been proposed previously (Childs, 1968). The relational model’s objectives were specified as follows:

- To allow a high degree of data independence. Application programs must not be affected by modifications to the internal data representation, particularly by changes to file organizations, record orderings, or access paths.
- To provide substantial grounds for dealing with data semantics, consistency, and redundancy problems. In particular, Codd’s paper introduced the concept of **normalized** relations, that is, relations that have no repeating groups. (The process of normalization is discussed in Chapters 13 and 14.)
- To enable the expansion of set-oriented data manipulation languages.

Although interest in the relational model came from several directions, the most significant research may be attributed to three projects with rather different perspectives. The first of these, at IBM’s San José Research Laboratory in California, was the prototype relational DBMS System R, which was developed during the late 1970s (Astrahan *et al.*, 1976). This project was designed to prove the practicality of the relational model by providing an implementation of its data structures and operations. It also proved to be an excellent source of information about implementation concerns such as transaction management, concurrency control, recovery techniques, query optimization, data security and integrity, human factors, and user interfaces, and led to the publication of many research papers and to the development of other prototypes. In particular, the System R project led to two major developments:

- the development of a structured query language called SQL (pronounced ‘S-Q-L’, or sometimes ‘See-Quel’), which has since become the formal International Organization for Standardization (ISO) and *de facto* standard language for relational DBMSs;
- the production of various commercial relational DBMS products during the late 1970s and the 1980s: for example, DB2 and SQL/DS from IBM and Oracle from Oracle Corporation.

The second project to have been significant in the development of the relational model was the INGRES (Interactive Graphics Retrieval System) project at the University of California at Berkeley, which was active at about the same time as the System R project. The INGRES project involved the development of a prototype RDBMS, with the research concentrating on the same overall objectives as the System R project. This research led to an academic version of INGRES, which contributed to the general appreciation of relational concepts, and spawned the commercial products INGRES from Relational Technology Inc. (now Advantage Ingres Enterprise Relational Database from Computer Associates) and the Intelligent Database Machine from Britton Lee Inc.

The third project was the Peterlee Relational Test Vehicle at the IBM UK Scientific Centre in Peterlee (Todd, 1976). This project had a more theoretical orientation than the System R and INGRES projects and was significant, principally for research into such issues as query processing and optimization, and functional extension.

Commercial systems based on the relational model started to appear in the late 1970s and early 1980s. Now there are several hundred RDBMSs for both mainframe and PC environments, even though many do not strictly adhere to the definition of the relational model. Examples of PC-based RDBMSs are Office Access and Visual FoxPro from Microsoft, InterBase and JDataStore from Borland, and R:Base from R:BASE Technologies.

Owing to the popularity of the relational model, many non-relational systems now provide a relational user interface, irrespective of the underlying model. Computer Associates’ IDMS, the principal network DBMS, has become Advantage CA-IDMS, supporting a relational view of data. Other mainframe DBMSs that support some relational features are Computer Corporation of America’s Model 204 and Software AG’s ADABAS.

Some extensions to the relational model have also been proposed; for example, extensions to:

- capture more closely the meaning of data (for example, Codd, 1979);
- support object-oriented concepts (for example, Stonebraker and Rowe, 1986);
- support deductive capabilities (for example, Gardarin and Valduriez, 1989).

We discuss some of these extensions in Chapters 25–28 on Object DBMSs.

Terminology

3.2

The relational model is based on the mathematical concept of a **relation**, which is physically represented as a **table**. Codd, a trained mathematician, used terminology taken from mathematics, principally set theory and predicate logic. In this section we explain the terminology and structural concepts of the relational model.

3.2.1 Relational Data Structure

Relation A relation is a table with columns and rows.

An RDBMS requires only that the database be perceived by the user as tables. Note, however, that this perception applies only to the logical structure of the database: that is, the external and conceptual levels of the ANSI-SPARC architecture discussed in Section 2.1. It does not apply to the physical structure of the database, which can be implemented using a variety of storage structures (see Appendix C).

Attribute An attribute is a named column of a relation.

In the relational model, **relations** are used to hold information about the objects to be represented in the database. A relation is represented as a two-dimensional table in which the rows of the table correspond to individual records and the table columns correspond to **attributes**. Attributes can appear in any order and the relation will still be the same relation, and therefore convey the same meaning.

For example, the information on branch offices is represented by the Branch relation, with columns for attributes branchNo (the branch number), street, city, and postcode. Similarly, the information on staff is represented by the Staff relation, with columns for attributes staffNo (the staff number), fName, lName, position, sex, DOB (date of birth), salary, and branchNo (the number of the branch the staff member works at). Figure 3.1 shows instances of the Branch and Staff relations. As you can see from this example, a column contains values of a single attribute; for example, the branchNo columns contain only numbers of existing branch offices.

Domain A domain is the set of allowable values for one or more attributes.

Domains are an extremely powerful feature of the relational model. Every attribute in a relation is defined on a **domain**. Domains may be distinct for each attribute, or two or more attributes may be defined on the same domain. Figure 3.2 shows the domains for some of the attributes of the Branch and Staff relations. Note that, at any given time, typically there will be values in a domain that do not currently appear as values in the corresponding attribute.

The domain concept is important because it allows the user to define in a central place the meaning and source of values that attributes can hold. As a result, more information is available to the system when it undertakes the execution of a relational operation, and operations that are semantically incorrect can be avoided. For example, it is not sensible to compare a street name with a telephone number, even though the domain definitions for both these attributes are character strings. On the other hand, the monthly rental on a property and the number of months a property has been leased have different domains (the first a monetary value, the second an integer value), but it is still a legal operation to

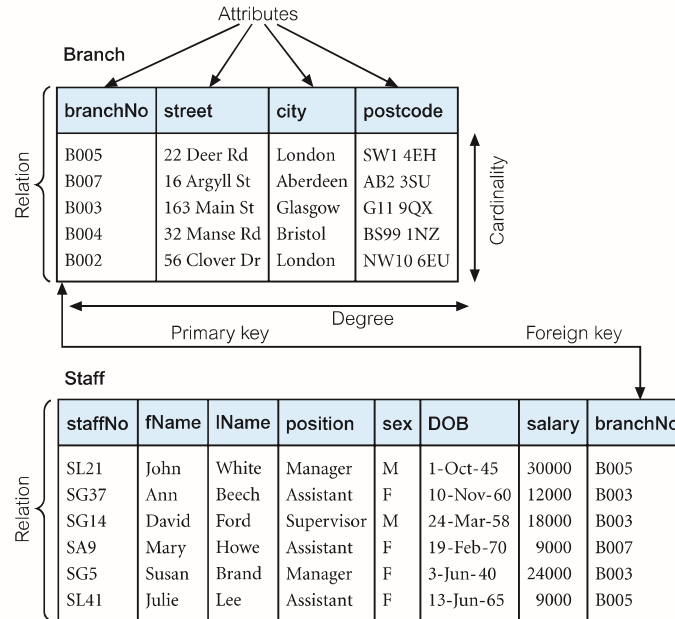


Figure 3.1
Instances of the
Branch and Staff
relations.

Attribute	Domain Name	Meaning	Domain Definition
branchNo	BranchNumbers	The set of all possible branch numbers	character: size 4, range B001–B999
street	StreetNames	The set of all street names in Britain	character: size 25
city	CityNames	The set of all city names in Britain	character: size 15
postcode	Postcodes	The set of all postcodes in Britain	character: size 8
sex	Sex	The sex of a person	character: size 1, value M or F
DOB	DatesOfBirth	Possible values of staff birth dates	date, range from 1-Jan-20, format dd-mmm-yy
salary	Salaries	Possible values of staff salaries	monetary: 7 digits, range 6000.00–40000.00

Figure 3.2
Domains for some
attributes of the
Branch and Staff
relations.

multiply two values from these domains. As these two examples illustrate, a complete implementation of domains is not straightforward and, as a result, many RDBMSs do not support them fully.

Tuple A tuple is a row of a relation.

The elements of a relation are the rows or **tuples** in the table. In the Branch relation, each row contains four values, one for each attribute. Tuples can appear in any order and the relation will still be the same relation, and therefore convey the same meaning.

The structure of a relation, together with a specification of the domains and any other restrictions on possible values, is sometimes called its **intension**, which is usually fixed unless the meaning of a relation is changed to include additional attributes. The tuples are called the **extension** (or **state**) of a relation, which changes over time.

Degree The degree of a relation is the number of attributes it contains.

The Branch relation in Figure 3.1 has four attributes or degree four. This means that each row of the table is a four-tuple, containing four values. A relation with only one attribute would have degree one and be called a **unary** relation or one-tuple. A relation with two attributes is called **binary**, one with three attributes is called **ternary**, and after that the term **n-ary** is usually used. The degree of a relation is a property of the *intension* of the relation.

Cardinality The cardinality of a relation is the number of tuples it contains.

By contrast, the number of tuples is called the **cardinality** of the relation and this changes as tuples are added or deleted. The cardinality is a property of the *extension* of the relation and is determined from the particular instance of the relation at any given moment. Finally, we have the definition of a relational database.

Relational database A collection of normalized relations with distinct relation names.

A relational database consists of relations that are appropriately structured. We refer to this appropriateness as *normalization*. We defer the discussion of normalization until Chapters 13 and 14.

Alternative terminology

The terminology for the relational model can be quite confusing. We have introduced two sets of terms. In fact, a third set of terms is sometimes used: a relation may be referred to as a **file**, the tuples as **records**, and the attributes as **fields**. This terminology stems from the fact that, physically, the RDBMS may store each relation in a file. Table 3.1 summarizes the different terms for the relational model.

Table 3.1 Alternative terminology for relational model terms.

Formal terms	Alternative 1	Alternative 2
Relation	Table	File
Tuple	Row	Record
Attribute	Column	Field

For each column, we may specify that the order is ascending (ASC) or descending (DESC), with ASC being the default setting. For example, if we create an index on the PropertyForRent table as:

```
CREATE INDEX RentInd ON PropertyForRent (city, rent);
```

then an index called RentInd is created for the PropertyForRent table. Entries will be in alphabetical order by city and then by rent within each city.

6.3.6 Removing an Index (DROP INDEX)

If we create an index for a base table and later decide that it is no longer needed, we can use the DROP INDEX statement to remove the index from the database. DROP INDEX has the format:

```
DROP INDEX IndexName
```

The following statement will remove the index created in the previous example:

```
DROP INDEX RentInd;
```

6.4 Views

Recall from Section 3.4 the definition of a view:

View The dynamic result of one or more relational operations operating on the base relations to produce another relation. A view is a *virtual relation* that does not necessarily exist in the database but can be produced upon request by a particular user, at the time of request.

To the database user, a view appears just like a real table, with a set of named columns and rows of data. However, unlike a base table, a view does not necessarily exist in the database as a stored set of data values. Instead, a view is defined as a query on one or more base tables or views. The DBMS stores the definition of the view in the database. When the DBMS encounters a reference to a view, one approach is to look up this definition and translate the request into an equivalent request against the source tables of the view and then perform the equivalent request. This merging process, called **view resolution**, is discussed in Section 6.4.3. An alternative approach, called **view materialization**, stores the view as a temporary table in the database and maintains the currency of the view as the underlying base tables are updated. We discuss view materialization in Section 6.4.8. First, we examine how to create and use views.

Creating a View (CREATE VIEW)

6.4.1

The format of the CREATE VIEW statement is:

```
CREATE VIEW ViewName [(newColumnName [, . . . ])]
AS subselect [WITH [CASCADED | LOCAL] CHECK OPTION]
```

A view is defined by specifying an SQL SELECT statement. A name may optionally be assigned to each column in the view. If a list of column names is specified, it must have the same number of items as the number of columns produced by the *subselect*. If the list of column names is omitted, each column in the view takes the name of the corresponding column in the *subselect* statement. The list of column names must be specified if there is any ambiguity in the name for a column. This may occur if the *subselect* includes calculated columns, and the AS subclause has not been used to name such columns, or it produces two columns with identical names as the result of a join.

The *subselect* is known as the **defining query**. If WITH CHECK OPTION is specified, SQL ensures that if a row fails to satisfy the WHERE clause of the defining query of the view, it is not added to the underlying base table of the view (see Section 6.4.6). It should be noted that to create a view successfully, you must have SELECT privilege on all the tables referenced in the subselect and USAGE privilege on any domains used in referenced columns. These privileges are discussed further in Section 6.6. Although all views are created in the same way, in practice different types of view are used for different purposes. We illustrate the different types of view with examples.

Example 6.3 Create a horizontal view

Create a view so that the manager at branch B003 can see only the details for staff who work in his or her branch office.

A horizontal view restricts a user's access to selected rows of one or more tables.

```
CREATE VIEW Manager3Staff
AS SELECT *
FROM Staff
WHERE branchNo = 'B003';
```

This creates a view called Manager3Staff with the same column names as the Staff table but containing only those rows where the branch number is B003. (Strictly speaking, the branchNo column is unnecessary and could have been omitted from the definition of the view, as all entries have branchNo = 'B003'.) If we now execute the statement:

```
SELECT * FROM Manager3Staff;
```

we would get the result table shown in Table 6.3. To ensure that the branch manager can see only these rows, the manager should not be given access to the base table Staff. Instead, the manager should be given access permission to the view Manager3Staff. This, in effect, gives the branch manager a customized view of the Staff table, showing only the staff at his or her own branch. We discuss access permissions in Section 6.6.

Table 6.3 Data for view Manager3Staff.

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000.00	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000.00	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000.00	B003

Example 6.4 Create a vertical view

Create a view of the staff details at branch B003 that excludes salary information, so that only managers can access the salary details for staff who work at their branch.

A vertical view restricts a user's access to selected columns of one or more tables.

```
CREATE VIEW Staff3
AS SELECT staffNo, fName, lName, position, sex
FROM Staff
WHERE branchNo = 'B003';
```

Note that we could rewrite this statement to use the Manager3Staff view instead of the Staff table, thus:

```
CREATE VIEW Staff3
AS SELECT staffNo, fName, lName, position, sex
FROM Manager3Staff;
```

Either way, this creates a view called Staff3 with the same columns as the Staff table, but excluding the salary, DOB, and branchNo columns. If we list this view we would get the result table shown in Table 6.4. To ensure that only the branch manager can see the salary details, staff at branch B003 should not be given access to the base table Staff or the view Manager3Staff. Instead, they should be given access permission to the view Staff3, thereby denying them access to sensitive salary data.

Vertical views are commonly used where the data stored in a table is used by various users or groups of users. They provide a private table for these users composed only of the columns they need.

Table 6.4 Data for view Staff3.

staffNo	fName	lName	position	sex
SG37	Ann	Beech	Assistant	F
SG14	David	Ford	Supervisor	M
SG5	Susan	Brand	Manager	F

Example 6.5 Grouped and joined views

Create a view of staff who manage properties for rent, which includes the branch number they work at, their staff number, and the number of properties they manage (see Example 5.27).

```
CREATE VIEW StaffPropCnt (branchNo, staffNo, cnt)
AS SELECT s.branchNo, s.staffNo, COUNT(*)
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
GROUP BY s.branchNo, s.staffNo;
```

This gives the data shown in Table 6.5. This example illustrates the use of a subselect containing a **GROUP BY** clause (giving a view called a **grouped view**), and containing multiple tables (giving a view called a **joined view**). One of the most frequent reasons for using views is to simplify multi-table queries. Once a joined view has been defined, we can often use a simple single-table query against the view for queries that would otherwise require a multi-table join. Note that we have to name the columns in the definition of the view because of the use of the unqualified aggregate function **COUNT** in the subselect.

Table 6.5 Data for view StaffPropCnt.

branchNo	staffNo	cnt
B003	SG14	1
B003	SG37	2
B005	SL41	1
B007	SA9	1

Removing a View (DROP VIEW)

6.4.2

A view is removed from the database with the **DROP VIEW** statement:

```
DROP VIEW ViewName [RESTRICT | CASCADE]
```

DROP VIEW causes the definition of the view to be deleted from the database. For example, we could remove the Manager3Staff view using the statement:

```
DROP VIEW Manager3Staff;
```

If **CASCADE** is specified, **DROP VIEW** deletes all related dependent objects, in other words, all objects that reference the view. This means that **DROP VIEW** also deletes any

views that are defined on the view being dropped. If **RESTRICT** is specified and there are any other objects that depend for their existence on the continued existence of the view being dropped, the command is rejected. The default setting is **RESTRICT**.

6.4.3 View Resolution

Having considered how to create and use views, we now look more closely at how a query on a view is handled. To illustrate the process of **view resolution**, consider the following query that counts the number of properties managed by each member of staff at branch office B003. This query is based on the `StaffPropCnt` view of Example 6.5:

```
SELECT staffNo, cnt
FROM StaffPropCnt
WHERE branchNo = 'B003'
ORDER BY staffNo;
```

View resolution merges the above query with the defining query of the `StaffPropCnt` view as follows:

- (1) The view column names in the **SELECT** list are translated into their corresponding column names in the defining query. This gives:

```
SELECT s.staffNo AS staffNo, COUNT(*) AS cnt
```

- (2) View names in the **FROM** clause are replaced with the corresponding **FROM** lists of the defining query:

```
FROM Staff s, PropertyForRent p
```

- (3) The **WHERE** clause from the user query is combined with the **WHERE** clause of the defining query using the logical operator **AND**, thus:

```
WHERE s.staffNo = p.staffNo AND branchNo = 'B003'
```

- (4) The **GROUP BY** and **HAVING** clauses are copied from the defining query. In this example, we have only a **GROUP BY** clause:

```
GROUP BY s.branchNo, s.staffNo
```

- (5) Finally, the **ORDER BY** clause is copied from the user query with the view column name translated into the defining query column name:

```
ORDER BY s.staffNo
```

- (6) The final merged query becomes:

```
SELECT s.staffNo AS staffNo, COUNT(*) AS cnt
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo AND branchNo = 'B003'
GROUP BY s.branchNo, s.staffNo
ORDER BY s.staffNo;
```

This gives the result table shown in Table 6.6.

Table 6.6 Result table after view resolution.

staffNo	cnt
SG14	1
SG37	2

Restrictions on Views

6.4.4

The ISO standard imposes several important restrictions on the creation and use of views, although there is considerable variation among dialects.

- If a column in the view is based on an aggregate function, then the column may appear only in **SELECT** and **ORDER BY** clauses of queries that access the view. In particular, such a column may not be used in a **WHERE** clause and may not be an argument to an aggregate function in any query based on the view. For example, consider the view `StaffPropCnt` of Example 6.5, which has a column `cnt` based on the aggregate function **COUNT**. The following query would fail:

```
SELECT COUNT(cnt)
FROM StaffPropCnt;
```

because we are using an aggregate function on the column `cnt`, which is itself based on an aggregate function. Similarly, the following query would also fail:

```
SELECT *
FROM StaffPropCnt
WHERE cnt > 2;
```

because we are using the view column, `cnt`, derived from an aggregate function in a **WHERE** clause.

- A grouped view may never be joined with a base table or a view. For example, the `StaffPropCnt` view is a grouped view, so that any attempt to join this view with another table or view fails.

View Updatability

6.4.5

All updates to a base table are immediately reflected in all views that encompass that base table. Similarly, we may expect that if a view is updated then the base table(s) will reflect that change. However, consider again the view `StaffPropCnt` of Example 6.5. Consider what would happen if we tried to insert a record that showed that at branch B003, staff member SG5 manages two properties, using the following insert statement:

```
INSERT INTO StaffPropCnt
VALUES ('B003', 'SG5', 2);
```

We have to insert two records into the `PropertyForRent` table showing which properties staff member SG5 manages. However, we do not know which properties they are; all we know is that this member of staff manages two properties. In other words, we do not know the corresponding primary key values for the `PropertyForRent` table. If we change the definition of the view and replace the count with the actual property numbers:

```
CREATE VIEW StaffPropList (branchNo, staffNo, propertyNo)
AS SELECT s.branchNo, s.staffNo, p.propertyNo
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo;
```

and we try to insert the record:

```
INSERT INTO StaffPropList
VALUES ('B003', 'SG5', 'PG19');
```

then there is still a problem with this insertion, because we specified in the definition of the `PropertyForRent` table that all columns except `postcode` and `staffNo` were not allowed to have nulls (see Example 6.1). However, as the `StaffPropList` view excludes all columns from the `PropertyForRent` table except the property number, we have no way of providing the remaining non-null columns with values.

The ISO standard specifies the views that must be updatable in a system that conforms to the standard. The definition given in the ISO standard is that a view is updatable if and only if:

- `DISTINCT` is not specified; that is, duplicate rows must not be eliminated from the query results.
- Every element in the `SELECT` list of the defining query is a column name (rather than a constant, expression, or aggregate function) and no column name appears more than once.
- The `FROM` clause specifies only one table; that is, the view must have a single source table for which the user has the required privileges. If the source table is itself a view, then that view must satisfy these conditions. This, therefore, excludes any views based on a join, union (`UNION`), intersection (`INTERSECT`), or difference (`EXCEPT`).
- The `WHERE` clause does not include any nested `SELECT`s that reference the table in the `FROM` clause.
- There is no `GROUP BY` or `HAVING` clause in the defining query.

In addition, every row that is added through the view must not violate the integrity constraints of the base table. For example, if a new row is added through a view, columns that are not included in the view are set to null, but this must not violate a `NOT NULL` integrity constraint in the base table. The basic concept behind these restrictions is as follows:

Updatable view

For a view to be updatable, the DBMS must be able to trace any row or column back to its row or column in the source table.

WITH CHECK OPTION

6.4.6

Rows exist in a view because they satisfy the WHERE condition of the defining query. If a row is altered such that it no longer satisfies this condition, then it will disappear from the view. Similarly, new rows will appear within the view when an insert or update on the view cause them to satisfy the WHERE condition. The rows that enter or leave a view are called **migrating rows**.

Generally, the WITH CHECK OPTION clause of the CREATE VIEW statement prohibits a row migrating out of the view. The optional qualifiers LOCAL/CASCADED are applicable to view hierarchies: that is, a view that is derived from another view. In this case, if WITH LOCAL CHECK OPTION is specified, then any row insert or update on this view, and on any view directly or indirectly defined on this view, must not cause the row to disappear from the view, unless the row also disappears from the underlying derived view/table. If the WITH CASCADED CHECK OPTION is specified (the default setting), then any row insert or update on this view and on any view directly or indirectly defined on this view must not cause the row to disappear from the view.

This feature is so useful that it can make working with views more attractive than working with the base tables. When an INSERT or UPDATE statement on the view violates the WHERE condition of the defining query, the operation is rejected. This enforces constraints on the database and helps preserve database integrity. The WITH CHECK OPTION can be specified only for an updatable view, as defined in the previous section.

Example 6.6 WITH CHECK OPTION

Consider again the view created in Example 6.3:

```
CREATE VIEW Manager3Staff  
AS SELECT *  
  FROM Staff  
  WHERE branchNo = 'B003'  
WITH CHECK OPTION;
```

with the virtual table shown in Table 6.3. If we now attempt to update the branch number of one of the rows from B003 to B005, for example:

```
UPDATE Manager3Staff  
SET branchNo = 'B005'  
WHERE staffNo = 'SG37';
```

then the specification of the WITH CHECK OPTION clause in the definition of the view prevents this from happening, as this would cause the row to migrate from this horizontal view. Similarly, if we attempt to insert the following row through the view:

```
INSERT INTO Manager3Staff  
VALUES('SL15', 'Mary', 'Black', 'Assistant', 'F', DATE'1967-06-21', 8000, 'B002');
```

then the specification of `WITH CHECK OPTION` would prevent the row from being inserted into the underlying `Staff` table and immediately disappearing from this view (as branch `B002` is not part of the view).

Now consider the situation where `Manager3Staff` is defined not on `Staff` directly but on another view of `Staff`:

```
CREATE VIEW LowSalary AS SELECT *
FROM Staff
WHERE salary > 9000;

CREATE VIEW HighSalary AS SELECT *
FROM LowSalary
WHERE salary > 10000
WITH LOCAL CHECK OPTION;

CREATE VIEW Manager3Staff AS SELECT *
FROM HighSalary
WHERE branchNo = 'B003';
```

If we now attempt the following update on `Manager3Staff`:

```
UPDATE Manager3Staff
SET salary = 9500
WHERE staffNo = 'SG37';
```

then this update would fail: although the update would cause the row to disappear from the view `HighSalary`, the row would not disappear from the table `LowSalary` that `HighSalary` is derived from. However, if instead the update tried to set the salary to 8000, then the update would succeed as the row would no longer be part of `LowSalary`. Alternatively, if the view `HighSalary` had specified `WITH CASCADED CHECK OPTION`, then setting the salary to either 9500 or 8000 would be rejected because the row would disappear from `HighSalary`. Therefore, to ensure that anomalies like this do not arise, each view should normally be created using the `WITH CASCADED CHECK OPTION`.

6.4.7 Advantages and Disadvantages of Views

Restricting some users' access to views has potential advantages over allowing users direct access to the base tables. Unfortunately, views in SQL also have disadvantages. In this section we briefly review the advantages and disadvantages of views in SQL as summarized in Table 6.7.

Table 6.7 Summary of advantages/disadvantages of views in SQL.

Advantages	Disadvantages
Data independence	Update restriction
Currency	Structure restriction
Improved security	Performance
Reduced complexity	
Convenience	
Customization	
Data integrity	

Advantages

In the case of a DBMS running on a standalone PC, views are usually a convenience, defined to simplify database requests. However, in a multi-user DBMS, views play a central role in defining the structure of the database and enforcing security. The major advantages of views are described below.

Data independence

A view can present a consistent, unchanging picture of the structure of the database, even if the underlying source tables are changed (for example, columns added or removed, relationships changed, tables split, restructured, or renamed). If columns are added or removed from a table, and these columns are not required by the view, then the definition of the view need not change. If an existing table is rearranged or split up, a view may be defined so that users can continue to see the old table. In the case of splitting a table, the old table can be recreated by defining a view from the join of the new tables, provided that the split is done in such a way that the original table can be reconstructed. We can ensure that this is possible by placing the primary key in both of the new tables. Thus, if we originally had a Client table of the form:

Client (clientNo, fName, lName, telNo, prefType, maxRent)

we could reorganize it into two new tables:

ClientDetails (clientNo, fName, lName, telNo)

ClientReqs (clientNo, prefType, maxRent)

Users and applications could still access the data using the old table structure, which would be recreated by defining a view called Client as the natural join of ClientDetails and ClientReqs, with clientNo as the join column:

```
CREATE VIEW Client
AS SELECT cd.clientNo, fName, lName, telNo, prefType, maxRent
FROM ClientDetails cd, ClientReqs cr
WHERE cd.clientNo = cr.clientNo;
```

Currency

Changes to any of the base tables in the defining query are immediately reflected in the view.

Improved security

Each user can be given the privilege to access the database only through a small set of views that contain the data appropriate for that user, thus restricting and controlling each user's access to the database.

Reduced complexity

A view can simplify queries, by drawing data from several tables into a single table, thereby transforming multi-table queries into single-table queries.

Convenience

Views can provide greater convenience to users as users are presented with only that part of the database that they need to see. This also reduces the complexity from the user's point of view.

Customization

Views provide a method to customize the appearance of the database, so that the same underlying base tables can be seen by different users in different ways.

Data integrity

If the `WITH CHECK OPTION` clause of the `CREATE VIEW` statement is used, then SQL ensures that no row that fails to satisfy the `WHERE` clause of the defining query is ever added to any of the underlying base table(s) through the view, thereby ensuring the integrity of the view.

Disadvantages

Although views provide many significant benefits, there are also some disadvantages with SQL views.

Update restriction

In Section 6.4.5 we showed that, in some cases, a view cannot be updated.

Structure restriction

The structure of a view is determined at the time of its creation. If the defining query was of the form `SELECT * FROM . . .`, then the `*` refers to the columns of the base table present when the view is created. If columns are subsequently added to the base table, then these columns will not appear in the view, unless the view is dropped and recreated.

Performance

There is a performance penalty to be paid when using a view. In some cases, this will be negligible; in other cases, it may be more problematic. For example, a view defined by a complex, multi-table query may take a long time to process as the view resolution must join the tables together *every time the view is accessed*. View resolution requires additional computer resources. In the next section, we briefly discuss an alternative approach to maintaining views that attempts to overcome this disadvantage.

6.4.8 View Materialization

In Section 6.4.3 we discussed one approach to handling queries based on a view, where the query is modified into a query on the underlying base tables. One disadvantage with this approach is the time taken to perform the view resolution, particularly if the view is accessed frequently. An alternative approach, called **view materialization**, is to store

inconsistencies and data loss. Many DBMSs allow users to undertake simultaneous operations on the database. If these operations are not controlled, the accesses may interfere with one another and the database can become inconsistent. To overcome this, the DBMS implements a **concurrency control** protocol that prevents database accesses from interfering with one another.

Database recovery is the process of restoring the database to a correct state following a failure. The failure may be the result of a system crash due to hardware or software errors, a media failure, such as a head crash, or a software error in the application, such as a logical error in the program that is accessing the database. It may also be the result of unintentional or intentional corruption or destruction of data or facilities by system administrators or users. Whatever the underlying cause of the failure, the DBMS must be able to recover from the failure and restore the database to a consistent state.

Structure of this Chapter

Central to an understanding of both concurrency control and recovery is the notion of a **transaction**, which we describe in Section 20.1. In Section 20.2 we discuss concurrency control and examine the protocols that can be used to prevent conflict. In Section 20.3 we discuss database recovery and examine the techniques that can be used to ensure the database remains in a consistent state in the presence of failures. In Section 20.4 we examine more advanced transaction models that have been proposed for transactions that are of a long duration (from hours to possibly even months) and have uncertain developments, so that some actions cannot be foreseen at the beginning. In Section 20.5 we examine how Oracle handles concurrency control and recovery.

In this chapter we consider transaction support, concurrency control, and recovery for a centralized DBMS, that is a DBMS that consists of a single database. Later, in Chapter 23, we consider these services for a distributed DBMS, that is a DBMS that consists of multiple logically related databases distributed across a network.

Transaction Support

20.1

Transaction An action, or series of actions, carried out by a single user or application program, which reads or updates the contents of the database.

A transaction is a **logical unit of work** on the database. It may be an entire program, a part of a program, or a single command (for example, the SQL command INSERT or UPDATE), and it may involve any number of operations on the database. In the database context, the execution of an application program can be thought of as one or more transactions with non-database processing taking place in between. To illustrate the concepts of a transaction, we examine two relations from the instance of the *DreamHome* rental database shown in Figure 3.3:

Figure 20.1

Example transactions.

<pre> read(staffNo = x, salary) salary = salary * 1.1 write(staffNo = x, salary) </pre>	<pre> delete(staffNo = x) for all PropertyForRent records, pno begin read(propertyNo = pno, staffNo) if (staffNo = x) then begin staffNo = newStaffNo write(propertyNo = pno, staffNo) end end end </pre>
(a)	(b)

Staff (staffNo, fName, lName, position, sex, DOB, salary, branchNo)

PropertyForRent (propertyNo, street, city, postcode, type, rooms, rent, ownerNo, staffNo, branchNo)

A simple transaction against this database is to update the salary of a particular member of staff given the staff number, x . At a high level, we could write this transaction as shown in Figure 20.1(a). In this chapter we denote a database read or write operation on a data item x as $\text{read}(x)$ or $\text{write}(x)$. Additional qualifiers may be added as necessary; for example, in Figure 20.1(a), we have used the notation $\text{read}(\text{staffNo} = x, \text{salary})$ to indicate that we want to read the data item *salary* for the tuple with primary key value x . In this example, we have a **transaction** consisting of two database operations (read and write) and a non-database operation ($\text{salary} = \text{salary} * 1.1$).

A more complicated transaction is to delete the member of staff with a given staff number x , as shown in Figure 20.1(b). In this case, as well as having to delete the tuple in the *Staff* relation, we also need to find all the *PropertyForRent* tuples that this member of staff managed and reassign them to a different member of staff, *newStaffNo* say. If all these updates are not made, referential integrity will be lost and the database will be in an **inconsistent state**: a property will be managed by a member of staff who no longer exists in the database.

A transaction should always transform the database from one consistent state to another, although we accept that consistency may be violated while the transaction is in progress. For example, during the transaction in Figure 20.1(b), there may be some moment when one tuple of *PropertyForRent* contains the new *newStaffNo* value and another still contains the old one, x . However, at the end of the transaction, all necessary tuples should have the new *newStaffNo* value.

A transaction can have one of two outcomes. If it completes successfully, the transaction is said to have **committed** and the database reaches a new consistent state. On the other hand, if the transaction does not execute successfully, the transaction is **aborted**. If a transaction is aborted, the database must be restored to the consistent state it was in before the transaction started. Such a transaction is **rolled back** or **undone**. A committed transaction cannot be aborted. If we decide that the committed transaction was a mistake, we must perform another **compensating transaction** to reverse its effects (as we discuss in Section 20.4.2). However, an aborted transaction that is rolled back can be restarted later and, depending on the cause of the failure, may successfully execute and commit at that time.

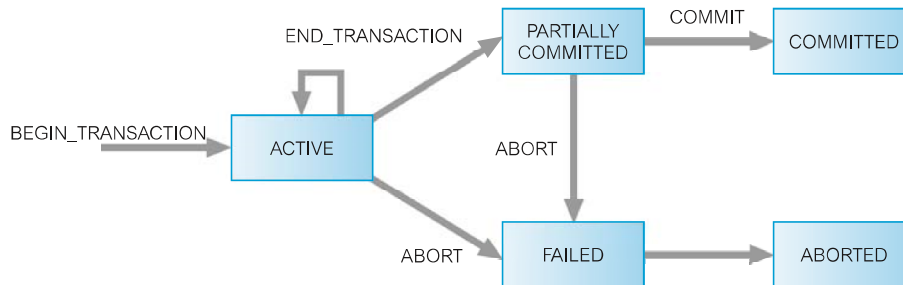


Figure 20.2
State transition
diagram for a
transaction.

The DBMS has no inherent way of knowing which updates are grouped together to form a single logical transaction. It must therefore provide a method to allow the user to indicate the boundaries of a transaction. The keywords `BEGIN TRANSACTION`, `COMMIT`, and `ROLLBACK` (or their equivalent[†]) are available in many data manipulation languages to delimit transactions. If these delimiters are not used, the entire program is usually regarded as a single transaction, with the DBMS automatically performing a `COMMIT` when the program terminates correctly and a `ROLLBACK` if it does not.

Figure 20.2 shows the state transition diagram for a transaction. Note that in addition to the obvious states of `ACTIVE`, `COMMITTED`, and `ABORTED`, there are two other states:

- **PARTIALLY COMMITTED**, which occurs after the final statement has been executed. At this point, it may be found that the transaction has violated serializability (see Section 20.2.2) or has violated an integrity constraint and the transaction has to be aborted. Alternatively, the system may fail and any data updated by the transaction may not have been safely recorded on secondary storage. In such cases, the transaction would go into the `FAILED` state and would have to be aborted. If the transaction has been successful, any updates can be safely recorded and the transaction can go to the `COMMITTED` state.
- **FAILED**, which occurs if the transaction cannot be committed or the transaction is aborted while in the `ACTIVE` state, perhaps due to the user aborting the transaction or as a result of the concurrency control protocol aborting the transaction to ensure serializability.

Properties of Transactions

20.1.1

There are properties that all transactions should possess. The four basic, or so-called **ACID**, properties of a transaction are (Haerder and Reuter, 1983):

- **Atomicity** The ‘all or nothing’ property. A transaction is an indivisible unit that is either performed in its entirety or is not performed at all. It is the responsibility of the recovery subsystem of the DBMS to ensure atomicity.
- **Consistency** A transaction must transform the database from one consistent state to another consistent state. It is the responsibility of both the DBMS and the application developers to ensure consistency. The DBMS can ensure consistency by enforcing all

[†] With the ISO SQL standard, `BEGIN TRANSACTION` is implied by the first *transaction-initiating* SQL statement (see Section 6.5).

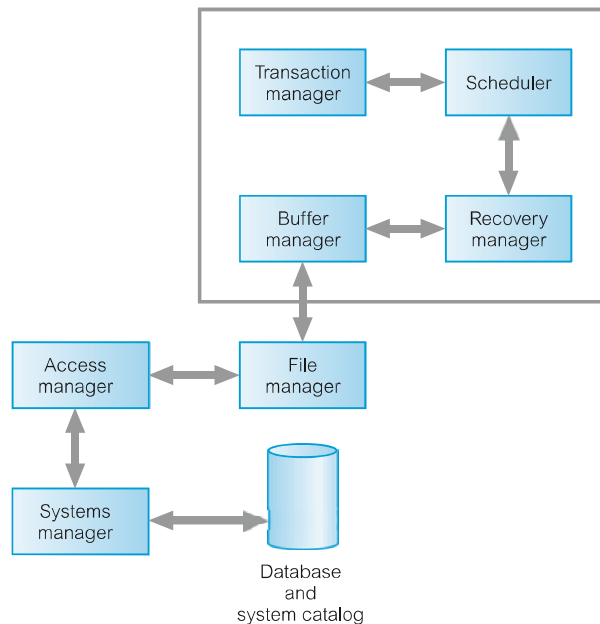
the constraints that have been specified on the database schema, such as integrity and enterprise constraints. However, in itself this is insufficient to ensure consistency. For example, suppose we have a transaction that is intended to transfer money from one bank account to another and the programmer makes an error in the transaction logic and debits one account but credits the wrong account, then the database is in an inconsistent state. However, the DBMS would not have been responsible for introducing this inconsistency and would have had no ability to detect the error.

- **Isolation** Transactions execute independently of one another. In other words, the partial effects of incomplete transactions should not be visible to other transactions. It is the responsibility of the concurrency control subsystem to ensure isolation.
- **Durability** The effects of a successfully completed (committed) transaction are permanently recorded in the database and must not be lost because of a subsequent failure. It is the responsibility of the recovery subsystem to ensure durability.

20.1.2 Database Architecture

In Chapter 2 we presented an architecture for a DBMS. Figure 20.3 represents an extract from Figure 2.8 identifying four high-level database modules that handle transactions, concurrency control, and recovery. The **transaction manager** coordinates transactions on behalf of application programs. It communicates with the **scheduler**, the module responsible for implementing a particular strategy for concurrency control. The scheduler is sometimes referred to as the **lock manager** if the concurrency control protocol is locking-based. The objective of the scheduler is to maximize concurrency without allowing

Figure 20.3
DBMS transaction
subsystem.



concurrently executing transactions to interfere with one another, and so compromise the integrity or consistency of the database.

If a failure occurs during the transaction, then the database could be inconsistent. It is the task of the **recovery manager** to ensure that the database is restored to the state it was in before the start of the transaction, and therefore a consistent state. Finally, the **buffer manager** is responsible for the efficient transfer of data between disk storage and main memory.

Concurrency Control

20.2

In this section we examine the problems that can arise with concurrent access and the techniques that can be employed to avoid these problems. We start with the following working definition of concurrency control.

Concurrency control The process of managing simultaneous operations on the database without having them interfere with one another.

The Need for Concurrency Control

20.2.1

A major objective in developing a database is to enable many users to access shared data concurrently. Concurrent access is relatively easy if all users are only reading data, as there is no way that they can interfere with one another. However, when two or more users are accessing the database simultaneously and at least one is updating data, there may be interference that can result in inconsistencies.

This objective is similar to the objective of multi-user computer systems, which allow two or more programs (or transactions) to execute at the same time. For example, many systems have input/output (I/O) subsystems that can handle I/O operations independently, while the main central processing unit (CPU) performs other operations. Such systems can allow two or more transactions to execute simultaneously. The system begins executing the first transaction until it reaches an I/O operation. While the I/O is being performed, the CPU suspends the first transaction and executes commands from the second transaction. When the second transaction reaches an I/O operation, control then returns to the first transaction and its operations are resumed from the point at which it was suspended. The first transaction continues until it again reaches another I/O operation. In this way, the operations of the two transactions are **interleaved** to achieve concurrent execution. In addition, **throughput** – the amount of work that is accomplished in a given time interval – is improved as the CPU is executing other transactions instead of being in an idle state waiting for I/O operations to complete.

However, although two transactions may be perfectly correct in themselves, the interleaving of operations in this way may produce an incorrect result, thus compromising the integrity and consistency of the database. We examine three examples of potential problems caused by concurrency: the **lost update problem**, the **uncommitted dependency problem**, and the **inconsistent analysis problem**. To illustrate these problems, we use a simple bank account relation that contains the *DreamHome* staff account balances. In this context, we are using the transaction as the *unit of concurrency control*.

Example 20.1 The lost update problem

An apparently successfully completed update operation by one user can be overridden by another user. This is known as the **lost update problem** and is illustrated in Figure 20.4, in which transaction T_1 is executing concurrently with transaction T_2 . T_1 is withdrawing £10 from an account with balance bal_x , initially £100, and T_2 is depositing £100 into the same account. If these transactions are executed **serially**, one after the other with no interleaving of operations, the final balance would be £190 no matter which transaction is performed first.

Transactions T_1 and T_2 start at nearly the same time, and both read the balance as £100. T_2 increases bal_x by £100 to £200 and stores the update in the database. Meanwhile, transaction T_1 decrements its copy of bal_x by £10 to £90 and stores this value in the database, overwriting the previous update, and thereby ‘losing’ the £100 previously added to the balance.

The loss of T_2 ’s update is avoided by preventing T_1 from reading the value of bal_x until after T_2 ’s update has been completed.

Figure 20.4
The lost update problem.

Time	T_1	T_2	bal_x
t_1		begin_transaction	100
t_2	begin_transaction	read(bal_x)	100
t_3	read(bal_x)	$bal_x = bal_x + 100$	100
t_4	$bal_x = bal_x - 10$	write(bal_x)	200
t_5	write(bal_x)	commit	90
t_6	commit		90

Example 20.2 The uncommitted dependency (or dirty read) problem

The uncommitted dependency problem occurs when one transaction is allowed to see the intermediate results of another transaction before it has committed. Figure 20.5 shows an example of an uncommitted dependency that causes an error, using the same initial value for balance bal_x as in the previous example. Here, transaction T_4 updates bal_x to £200,

Figure 20.5
The uncommitted dependency problem.

Time	T_3	T_4	bal_x
t_1		begin_transaction	100
t_2		read(bal_x)	100
t_3		$bal_x = bal_x + 100$	100
t_4	begin_transaction	write(bal_x)	200
t_5	read(bal_x)	:	200
t_6	$bal_x = bal_x - 10$	rollback	100
t_7	write(bal_x)		190
t_8	commit		190

but it aborts the transaction so that bal_x should be restored to its original value of £100. However, by this time transaction T_3 has read the new value of bal_x (£200) and is using this value as the basis of the £10 reduction, giving a new incorrect balance of £190, instead of £90. The value of bal_x read by T_3 is called *dirty data*, giving rise to the alternative name, *the dirty read problem*.

The reason for the rollback is unimportant; it may be that the transaction was in error, perhaps crediting the wrong account. The effect is the assumption by T_3 that T_4 's update completed successfully, although the update was subsequently rolled back. This problem is avoided by preventing T_3 from reading bal_x until after the decision has been made to either commit or abort T_4 's effects.

The two problems in these examples concentrate on transactions that are updating the database and their interference may corrupt the database. However, transactions that only read the database can also produce inaccurate results if they are allowed to read partial results of incomplete transactions that are simultaneously updating the database. We illustrate this with the next example.

Example 20.3 The inconsistent analysis problem

The problem of inconsistent analysis occurs when a transaction reads several values from the database but a second transaction updates some of them during the execution of the first. For example, a transaction that is summarizing data in a database (for example, totaling balances) will obtain inaccurate results if, while it is executing, other transactions are updating the database. One example is illustrated in Figure 20.6, in which a summary transaction T_6 is executing concurrently with transaction T_5 . Transaction T_6 is totaling the balances of account x (£100), account y (£50), and account z (£25). However, in the meantime, transaction T_5 has transferred £10 from bal_x to bal_z , so that T_6 now has the wrong result (£10 too high). This problem is avoided by preventing transaction T_6 from reading bal_x and bal_z until after T_5 has completed its updates.

Time	T_5	T_6	bal_x	bal_y	bal_z	sum
t_1		begin_transaction	100	50	25	
t_2	begin_transaction	sum = 0	100	50	25	0
t_3	read(bal_x)	read(bal_x)	100	50	25	0
t_4	$bal_x = bal_x - 10$	sum = sum + bal_x	100	50	25	100
t_5	write(bal_x)	read(bal_y)	90	50	25	100
t_6	read(bal_z)	sum = sum + bal_y	90	50	25	150
t_7	$bal_z = bal_z + 10$		90	50	25	150
t_8	write(bal_z)		90	50	35	150
t_9	commit	read(bal_z)	90	50	35	150
t_{10}		sum = sum + bal_z	90	50	35	185
t_{11}		commit	90	50	35	185

Figure 20.6
The inconsistent analysis problem.

Another problem can occur when a transaction *T* rereads a data item it has previously read but, in between, another transaction has modified it. Thus, *T* receives two different values for the same data item. This is sometimes referred to as a **nonrepeatable** (or **fuzzy**) **read**. A similar problem can occur if transaction *T* executes a query that retrieves a set of tuples from a relation satisfying a certain predicate, re-executes the query at a later time but finds that the retrieved set contains an additional (**phantom**) tuple that has been inserted by another transaction in the meantime. This is sometimes referred to as a **phantom read**.

20.2.2 Serializability and Recoverability

The objective of a concurrency control protocol is to schedule transactions in such a way as to avoid any interference between them, and hence prevent the types of problem described in the previous section. One obvious solution is to allow only one transaction to execute at a time: one transaction is *committed* before the next transaction is allowed to *begin*. However, the aim of a multi-user DBMS is also to maximize the degree of concurrency or parallelism in the system, so that transactions that can execute without interfering with one another can run in parallel. For example, transactions that access different parts of the database can be scheduled together without interference. In this section, we examine serializability as a means of helping to identify those executions of transactions that are *guaranteed* to ensure consistency (Papadimitriou, 1979). First, we give some definitions.

Schedule A sequence of the operations by a set of concurrent transactions that preserves the order of the operations in each of the individual transactions.

A transaction comprises a sequence of operations consisting of read and/or write actions to the database, followed by a commit or abort action. A schedule *S* consists of a sequence of the operations from a set of *n* transactions $T_1, T_2, \dots, T_n$, subject to the constraint that the order of operations for each transaction is preserved in the schedule. Thus, for each transaction T_i in schedule *S*, the order of the operations in T_i must be the same in schedule *S*.

Serial schedule A schedule where the operations of each transaction are executed consecutively without any interleaved operations from other transactions.

In a serial schedule, the transactions are performed in serial order. For example, if we have two transactions T_1 and T_2 , serial order would be T_1 followed by T_2 , or T_2 followed by T_1 . Thus, in serial execution there is no interference between transactions, since only one is executing at any given time. However, there is no guarantee that the results of all serial executions of a given set of transactions will be identical. In banking, for example, it matters whether interest is calculated on an account before a large deposit is made or after.

Nonserial schedule A schedule where the operations from a set of concurrent transactions are interleaved.